

Oracle Connect

Follow: 

Ideas for a competitive edge



Database Developer, Database, SQL and PL SQL, DBA, SQL 101

Why WHERE Matters

March 1, 2012 | 11 minute read



Profile

Part 4 in a series on the basics of the relational database and SQL

By Melanie Caffrey

March/April 2012

Part 3 in this series, “[Getting Answers with SELECT Statements](#)” (*Oracle Magazine*, January/February 2012), introduced the anatomy of a SELECT statement (or query) and the importance of ascertaining which tables contain data of interest. Now that you’re familiar with a SELECT statement’s basic functionality, you can start filtering your data to limit the output in meaningful ways. The WHERE clause enables you to narrow the scope of the data a SELECT statement retrieves (or *fetches*). WHERE, and its associated comparison and logical operators, are the focus of this article.

To try out the examples in this and subsequent articles in the series, you need access to an Oracle Database instance. If necessary, download and install an [Oracle Database edition](#) for your operating system. I recommend installing Oracle Database, Express Edition.

If you install the Oracle Database software, choose the installation option that enables you to create and configure a database. A new database, including sample user accounts and their associated *schemas*, will be created for you. (Recall from [Part 1](#) of this series that a schema is typically a grouping of objects, such as tables, that serve a similar business function.) SQL_101 is the user account you’ll use for the examples in this article; it’s also the schema in which you will create database tables and other objects. When the installation process prompts you to specify schema passwords, enter and confirm passwords for SYS and SYSTEM and make a note of them. Finally, whether you installed the database software

from scratch or have access to an existing Oracle Database instance, download and unzip the [SQL script](#) and run it to create the example tables for the SQL_101 schema.

The SQL queries in this article are executed against tables in the SQL_101 schema with the SQL*Plus tool.

Setting Limits by Comparing

To filter the data a query retrieves, you add a WHERE clause—also called a *predicate list* or a *set of conditions*—to your SQL statement. In a nutshell, the WHERE clause specifies criteria that must be met before records are included in your query result set. The WHERE clause must specify a WHERE clause *condition* (or conditions) that the database software evaluates to be true or false—alternatively, the software can determine the absence of a value. A WHERE clause consists of conditional *expressions*. A conditional expression takes the form

<left_expression> <as compared with> <right_expression>

Here are some examples of common types of conditional expressions:

 [Copy code snippet](#)

```
WHERE <column_name> =  
<literal_character_value>  
WHERE <column_name> IN (3, 7, 9)  
WHERE <column_name> >= 100  
WHERE <column_name> LIKE 'E%';  
WHERE <column_name> BETWEEN 100 AND 500;
```

A literal character value, or *string*, is any list of alphanumeric characters enclosed in single quotation marks, such as 'Smith', '73abc', or '15-MAR-1965'.

Comparison operators compare expressions to determine the appropriate data for selection. **Table 1** shows commonly used comparison operators.

Operator	Definition	Example
=	Equal	WHERE last_name = 'Michaels'
!= <>	Not equal Not equal	WHERE salary <> 100000
> >=	Greater than Greater than or equal to	WHERE salary >= 70000
< <=	Less than Less than or equal to	WHERE salary <= 85000
IN (...)	List of values	WHERE SALARY IN (70000, 85000, 100000)

BETWEEN ... AND ...	Inclusive of two values (and all values between them)	WHERE SALARY BETWEEN 70000 and 100000
LIKE	Does pattern matching with wildcard characters % and _	WHERE first_name LIKE 'F%'
IS NULL IS NOT NULL	Tests for null values Tests for non-null values	WHERE manager IS NULL

Table 1: SQL WHERE clause comparison operators

The Importance of (In)equality

The most commonly used comparison operator is the *equality* operator, =. For example, if you wanted to find out the names and hire dates of all employees with an annual salary of \$70,000, you could execute the SQL query in **Listing 1**.

Code Listing 1: Query for finding employees whose salary equals \$70,000

 [Copy code snippet](#)

```
SQL> select first_name, last_name, hire_date, salary
2      from employee
3      where salary = 70000;
FIRST_NAME  LAST_NAME      HIRE_DATE      SALARY
-----
Matthew     Michaels        16-MAY-07      70000
1 row selected.
```

The value stored in the SALARY column is compared with the literal value 70000 to determine whether the values are equal. Each row that satisfies the WHERE clause condition is retrieved.

Sometimes you might want to exclude certain data from your query results. For example, after the query and result in **Listing 1**, you already know the name, hire date, and salary of the employee named Matthew Michaels. To get the same information for all other employees, you could execute the query in **Listing 2**. As you can see, the query uses the inequality operator, !=, and retrieves every row except the one with the LAST_NAME value of Michaels.

Code Listing 2: Query that excludes the employee Michaels

 [Copy code snippet](#)

```
SQL> select first_name, last_name, hire_date, salary
2      from employee
3      where last_name != 'Michaels';
FIRST_NAME  LAST_NAME      HIRE_DATE      SALARY
-----
Frances     Newton         14-SEP-05      75000
Emily       Eckhardt       07-JUL-04      100000
Donald      Newton         24-SEP-06      80000
3 rows selected.
```

Be aware that when you compare a database column value with a character literal, or string, the case of the data contained in the database column must, by default, exactly match the case of the data contained in the string. The query in **Listing 3** returns no rows, because the case of the string denoting the employee's last name is different from that of the data stored in the EMPLOYEE table's LAST_NAME column.

Code Listing 3: Query using a literal value (case-sensitive) in a WHERE clause condition

 [Copy code snippet](#)

```
SQL> select first_name, last_name, hire_date, salary
2      from employee
3      where last_name = 'MICHAELS';
no rows selected
```

You'll learn about converting string literal data to match the case of data contained in a database column (and vice versa) in subsequent articles in this series.

Note in the example in **Listing 3** that when you compare a string literal with a database column value, you must enclose the string literal value in single quotation marks. The same requirement is true for comparing date literals with database column values.

Any two values you compare with each other must be of the same datatype. Compare only numbers with numbers, strings with strings, and dates with dates. Whenever possible, Oracle Database will perform an implicit datatype conversion, but in general, you should avoid allowing Oracle Database to do so. The query in **Listing 4** will return a result, but as a best practice, you should never compare a number with a string.

Code Listing 4: Query that performs an implicit datatype conversion

 [Copy code snippet](#)

```
SQL> select first_name, last_name, hire_date, salary
2      from employee
3      where salary = '70000';
FIRST_NAME  LAST_NAME      HIRE_DATE      SALARY
-----
Matthew     Michaels       16-MAY-07      70000
1 row selected.
```

The Range of Inclusion

Sometimes you need to obtain a set of records (rows) that falls within a certain range of values. You can do so with the BETWEEN operator, as in **Listing 5**.

The results of a BETWEEN operation can include the listed values that define the range. Therefore, in the example in **Listing 5**, the result list includes an employee with a salary of \$75,000, the lower end of the range, along with one whose salary of \$80,000 is between the upper and lower listed values. The BETWEEN operator is used most often for number and date comparisons.

Code Listing 5: Query for selecting records within a range of values

 [Copy code snippet](#)

```
SQL> select first_name, last_name, salary
       2   from employee
       3   where salary BETWEEN 75000 and 85000;
FIRST_NAME  LAST_NAME      SALARY
-----
Frances      Newton         75000
Donald       Newton         80000
2 rows selected.
```

The Greater and the Lesser

The comparison operators `>`, `>=`, `<`, and `<=` are useful if you need to obtain a set of records that fall either above or below certain criteria. In **Listing 6**, the *less than or equal to* operator, `<=`, is used to fetch a list of employees whose yearly salary is less than or equal to \$80,000.

Code Listing 6: Query using less than or equal to operator

 [Copy code snippet](#)

```
SQL> select first_name, last_name, salary
       2   from employee
       3   where salary <= 80000;
FIRST_NAME  LAST_NAME      SALARY
-----
Frances      Newton         75000
Donald       Newton         80000
Matthew      Michaels        70000
3 rows selected.
```

Match What You Like

Whenever you don't know or remember the exact spelling of a data value such as a name or you suspect data corruption (incorrect values in your database), you may want to perform an inexact search. The LIKE operator can help you carry out such a task. This operator performs pattern matching by using wildcard characters. The underscore (`_`) wildcard denotes a single character, and the percentage (`%`) wildcard denotes any number of characters (including zero characters). The query in **Listing 7** obtains records in which the last name begins with the uppercase letter *N* and contains the lowercase letter *w*. In the query in **Listing 7**, an unknown number of characters can exist between the *N* and the *w*, and an unknown number of characters can exist after the *w*—hence the use of two `%` wildcards in the expression.

Code Listing 7: Query using LIKE operator with literal string and wildcard values

 [Copy code snippet](#)

```
SQL> select first_name, last_name, salary
       2   from employee
       3   where last_name like 'N%w%';
FIRST_NAME  LAST_NAME      SALARY
-----
Frances      Newton         75000
```

```
Donald      Newton      80000
2 rows selected.
```

Consider the query in **Listing 8**. In this example, the WHERE clause limits the result set to rows in which the last name begins with two characters, has a lowercase letter *w* as the third character, and ends with any character(s) or at the third character. You can place the % or _ wildcard character anywhere within a literal character string (which, as always, must be enclosed in single quotation marks).

Code Listing 8: Query using LIKE operator with wildcard and literal values

 [Copy code snippet](#)

```
SQL> select first_name, last_name
2      from employee
3      where last_name like '__w%';
FIRST_NAME  LAST_NAME
-----
Frances     Newton
Donald      Newton
2 rows selected.
```

The IN Crowd

The IN operator evaluates a comma-delimited list of values enclosed within a set of parentheses. The query in **Listing 9** uses the IN operator to retrieve employees who have an annual salary of \$75,000, \$85,000, or \$100,000.

Code Listing 9: Query using IN operator with a list of values

 [Copy code snippet](#)

```
SQL> select first_name, last_name, salary
2      from employee
3      where salary in (75000, 85000, 100000);
FIRST_NAME  LAST_NAME      SALARY
-----
Frances     Newton         75000
Emily       Eckhardt       100000
2 rows selected.
```

Negating with NOT

The BETWEEN, IN, and LIKE comparison operators can all be negated with the NOT *logical operator*. (I'll describe logical operators shortly.) Consider the query in **Listing 10**. This query returns all the employees whose last name does *not* begin with an uppercase letter *N*.

Code Listing 10: Query using NOT and LIKE operators

 [Copy code snippet](#)

```
SQL> select first_name, last_name
2      from employee
3      where last_name NOT LIKE 'N%';
```

```

FIRST_NAME  LAST_NAME
-----
Emily       Eckhardt
Matthew     Michaels
2 rows selected.

```

Existence or Absence of Values

Recall from Part 1 in this series that the absence of a value is referred to as a *null* value. A null value cannot be equal or unequal to another null value or to any non-null value. Therefore, you must always use the IS NULL or IS NOT NULL comparison operators to evaluate whether a data value is null or not. For example, the query in **Listing 11** returns employees who do not yet have an assigned manager.

Code Listing 11: Query using IS NULL operator

 [Copy code snippet](#)

```

SQL> select first_name, last_name, manager
      2      from employee
      3      where manager IS NULL;
FIRST_NAME  LAST_NAME      MANAGER
-----
Frances     Newton
Emily       Eckhardt
2 rows selected.

```

Note that the DISTINCT keyword (which you learned about in Part 3 of this series) recognizes and returns NULL values:

 [Copy code snippet](#)

```

SQL> select DISTINCT manager
      2      from employee;
MANAGER
-----
28
2 rows selected.

```

To eliminate null values from a result set derived from a query that uses the DISTINCT keyword in its SELECT list, you can use the IS NOT NULL operator in your WHERE clause:

 [Copy code snippet](#)

```

SQL> select DISTINCT manager
      2      from employee
      3      where manager IS NOT NULL;
MANAGER
-----

```

28
1 row selected.

Truth in Logic

WHERE clauses with only one predicate are rare. The logical operators AND and OR are used to group multiple predicates contained within the same WHERE clause of a single SQL statement. Each added predicate further filters your result set. If two conditions are combined via the AND operator, both conditions must evaluate to true to produce a result. If two conditions are combined with the OR operator, only one of the conditions needs to evaluate to true to yield a result.

For example, the SQL statement in **Listing 12** combines two comparison operators by using the AND logical operator. The result displays employees who do not have an assigned manager (according to the EMPLOYEE table) and whose salary is greater than \$75,000.

Code Listing 12: Query using AND logical operator to combine multiple predicates

 [Copy code snippet](#)

```
SQL> select first_name, last_name, manager, salary
2      from employee
3      where salary > 75000
4            AND manager IS NULL;
FIRST_NAME  LAST_NAME    MANAGER      SALARY
-----
Emily       Eckhardt          100000
1 row selected.
```

Using the OR logical operator instead of the AND operator changes the result set to include two more rows, as shown in **Listing 13**.

Code Listing 13: Query using OR logical operator to combine multiple predicates

 [Copy code snippet](#)

```
SQL> select first_name, last_name, manager, salary
2      from employee
3      where salary > 75000
4            OR manager IS NULL;
FIRST_NAME  LAST_NAME    MANAGER      SALARY
-----
Frances     Newton          75000
Emily       Eckhardt          100000
Donald      Newton        28           80000
3 rows selected.
```

Logical Precedence

If you use both AND and OR in a WHERE clause, the AND operator will always take precedence over the OR operator. That is, any AND conditions are evaluated first.

Consider the SQL query in **Listing 14**.

Code Listing 14: Query using AND and OR logical operators [Copy code snippet](#)

```

SQL> select first_name, last_name, manager, salary
2     from employee
3     where manager IS NULL
4         AND salary    = 75000
5         OR salary    = 80000;
FIRST_NAME  LAST_NAME    MANAGER      SALARY
-----
Frances      Newton
Donald      Newton      28           80000
2 rows selected.

```

You can change the precedence of logical operators in the WHERE clause by grouping the expressions with parentheses. The query in **Listing 15** yields a different result from the preceding one because the OR condition in parentheses is evaluated before the AND condition.

Code Listing 15: Query using AND and OR logical operators with parenthetical precedence [Copy code snippet](#)

```

SQL> select first_name, last_name, manager, salary
2     from employee
3     where manager IS NULL
4         AND (salary = 75000
5         OR salary = 80000);
FIRST_NAME  LAST_NAME    MANAGER      SALARY
-----
Frances      Newton
1 row selected.

```

In this new query, *both* expressions—the manager is null AND the salary is either \$75,000 or \$80,000—must evaluate to true to produce a result. Because the record for Donald Newton satisfies the second condition but not the first, it is not in the result set.

If you write a predicate that contains a mixture of ANDs and ORs, I strongly recommend that you use parentheses to mandate the order of operation explicitly. In general, this practice will make your SQL more understandable, maintainable, and correct.

Conclusion

Only rarely will you write a query without a WHERE clause, and this article has shown you how to use the WHERE clause to expand upon simple SQL SELECT statements and filter data of interest to receive a more meaningful result set. You've seen how comparison operators are used in conjunction with the WHERE clause to help you specify your desired result. You've also seen how logical operators can be used to further filter your data by grouping predicates.

The next installment of SQL 101 will examine the ORDER BY clause of a SQL statement and take a closer look at Oracle's SQL*Plus tool.

Next Steps

READ more about relational database design, concepts, and SQL

- [Oracle Database Concepts 11g Release 2 \(11.2\)](#)
- [Oracle Database SQL Language Reference 11g Release 1 \(11.1\)](#)
- [Oracle SQL Developer User's Guide Release 2.1](#)

- READ [SQL 101 Parts 1-3](#)

- DOWNLOAD [the sample script for this article](#)

Photography by Ricardo Gomez, Unsplash

 Profile picture of

[< Previous post](#)

[Next post >](#)

Getting Answers with SELECT

[Melanie Caffrey](#) | 2 minute read

An Order of Sorts

[Melanie Caffrey](#) | 2 minute read

Resources for

About
Careers
Developers
Investors
Partners
Startups

Why Oracle

Analyst Reports
Best CRM
Cloud Economics
Corporate Responsibility
Security Practices

Learn

What is Customer Service?
What is ERP?
What is Marketing Automation?
What is Procurement?
What is Talent Management?
What is VM?

What's New

Try Oracle Cloud Free Tier
Oracle Sustainability
Oracle COVID-19 Response
Oracle and SailGP
Oracle and Premier League
Oracle and Red Bull Racing Honda

Contact Us

US Sales
1.800.633.0738
How can we help?
Subscribe to Oracle Content
Try Oracle Cloud Free Tier
Events
News